

Advanced Storage Solutions

Stratis is one of the new features that ships with **RHEL 8** distribution. **Stratis** is a local storage management solution that focuses on simplicity and improved usability while at the same time providing access to advanced storage features. It uses the XFS file system and grants you access to advanced storage capabilities such as:

- Thin provisioning
- File system snapshots
- Tiering
- Pool-based management
- Monitoring

Basically, **Stratis** is a storage pool that is created from one or more local disks or disk partitions. Stratis helps a System administrator set up and manage complex storage configurations.

Before we proceed further, let us have a look at some of the technical terms you are bound to bump into as we go along:

- **pool:** A pool is made up of one or more block devices. The total size of the pool is equal to the sum total of the block devices.
- **blockdev:** As you might have guessed it this refers to block devices such as disk partitions.
- **Filesystem:** A filesystem is a thinly provisioned layer which does not come as a total fixed size. The filesystem's actual size grows as data is added. Stratis automatically grows the size of the filesystem as data size nears the virtual size of the file system.

Block devices that you can use with Stratis include:

1. [LVM Logical Volumes](#)
2. [LUKS](#)
3. SSDs (Solid State Drives)
4. Device Mapper Multipath
5. [iSCSI](#)
6. HDDs (Hard Disk Drives)
7. mdraid
8. NVMe storage devices

Software Components of Stratis

Stratis provides 2 software utilities:

- **Stratis-cli:** This is the command-line tool that ships with Stratis.
- **Stratisd daemon:** This is a daemon that creates and manages block devices and plays a role in providing a DBUS API.

How to Install Stratis on RHEL 8

Having looked at what **Stratis** is and defined a few terminologies. Let's now install and configure **Stratis** on **RHEL 8** distribution (also works on **CentOS 8**).

Install Stratis on RHEL 8

Let's see how you can install **Stratis** on your **RHEL 8** system, log in as root user and run the command.

```
# yum install stratisd stratis-cli
```

```
[root@rhel-8 ~]# dnf install stratisd stratis-cli
Last metadata expiration check: 0:00:17 ago on Wed 23 Oct 2019 11:04:20 PM UTC.
Dependencies resolved.
```

Package	Arch	Version	Repository	Size
Installing:				
stratis-cli	noarch	1.0.2-1.el8	rhui-rhel-8-appstream-rhui-rpms	51 k
stratisd	x86_64	1.0.3-1.el8	rhui-rhel-8-appstream-rhui-rpms	1.2 M
Installing dependencies:				
python3-dbus-python-client-gen	noarch	0.6-2.el8	rhui-rhel-8-appstream-rhui-rpms	24 k
python3-into-dbus-python	noarch	0.06-2.el8	rhui-rhel-8-appstream-rhui-rpms	27 k
python3-dbus-signature-pyparsing	noarch	0.03-2.el8	rhui-rhel-8-appstream-rhui-rpms	19 k
python3-dbus-client-gen	noarch	0.4-1.el8	rhui-rhel-8-appstream-rhui-rpms	26 k
python3-justbytes	noarch	0.11-2.el8	rhui-rhel-8-appstream-rhui-rpms	40 k
python3-justbases	noarch	0.9-6.el8	rhui-rhel-8-appstream-rhui-rpms	45 k
device-mapper-persistent-data	x86_64	0.7.6-1.el8	rhui-rhel-8-baseos-rhui-rpms	452 k

```
Transaction Summary
-----
Install 9 Packages

Total download size: 1.9 M
Installed size: 6.0 M
Is this ok [y/N]:
```

To find more information about the [installed packages](#) run the command.

```
# rpm -qi stratisd stratis-cli
```

```
[root@rhel-8 ~]# rpm -qi stratisd stratis-cli
Name       : stratisd
Version    : 1.0.3
Release    : 1.el8
Architecture: x86_64
Install Date: Wed 23 Oct 2019 11:05:37 PM UTC
Group      : Unspecified
Size       : 4215640
License    : MPLv2.0
Signature  : RSA/SHA256, Mon 14 Jan 2019 08:33:12 PM UTC, Key ID 199e2f91fd431d51
Source RPM : stratisd-1.0.3-1.el8.src.rpm
Build Date : Fri 11 Jan 2019 04:50:20 AM UTC
Build Host : x86-vm-07.build.eng.bos.redhat.com
Relocations : (not relocatable)
Packager   : Red Hat, Inc. <http://bugzilla.redhat.com/bugzilla>
Vendor     : Red Hat, Inc.
URL        : https://github.com/stratis-storage/stratisd
Summary    : Daemon that manages block devices to create filesystems
Description :
Daemon that manages block devices to create filesystems.
Name       : stratis-cli
Version    : 1.0.2
Release    : 1.el8
Architecture: noarch
Install Date: Wed 23 Oct 2019 11:05:38 PM UTC
Group      : Unspecified
Size       : 129260
License    : ASL 2.0
Signature  : RSA/SHA256, Fri 14 Dec 2018 04:35:00 AM UTC, Key ID 199e2f91fd431d51
Source RPM : stratis-cli-1.0.2-1.el8.src.rpm
Build Date : Tue 11 Dec 2018 11:26:04 PM UTC
Build Host : x86-vm-08.build.eng.bos.redhat.com
Relocations : (not relocatable)
Packager   : Red Hat, Inc. <http://bugzilla.redhat.com/bugzilla>
Vendor     : Red Hat, Inc.
URL        : https://github.com/stratis-storage/stratis-cli
Summary    : Command-line tool for interacting with the Stratis daemon
Description :
stratis provides a command-line interface (CLI) for
interacting with the Stratis daemon, stratisd. stratis
interacts with stratisd via D-Bus.
```

After the successful installation of **Stratis**, start the service by running the command.

```
# systemctl enable --now stratisd
```

To check the status of **Stratis**, run the command.

```
# systemctl status stratisd
```

```
[root@rhel-8 ~]#
[root@rhel-8 ~]# systemctl enable --now stratisd
[root@rhel-8 ~]#
[root@rhel-8 ~]# systemctl status stratisd
● stratisd.service - A daemon that manages a pool of block devices to create flexible file systems
   Loaded: loaded (/usr/lib/systemd/system/stratisd.service; enabled; vendor preset: enabled)
   Active: active (running) since Wed 2019-10-23 23:26:49 UTC; 15s ago
     Docs: man:stratisd(8)
  Main PID: 1044 (stratisd)
    Tasks: 1 (limit: 4998)
   Memory: 800.0K
    CGroup: /system.slice/stratisd.service
            └─1044 /usr/libexec/stratisd --debug
```

Create a Stratis Pool

To create a **Stratis** pool you need block devices that are not in use or mounted. Also, it is assumed that **Stratisd** service is up and running. Additionally, the block devices that you are going to use need to be at least **1 GB** in size.

On our **RHEL 8** system, we have four additional block devices: `/dev/xvdb`, `/dev/xvdc`, `/dev/xvdd`, `/dev/xvde`. To display the block devices, run the [lsblk command](#).

```
# lsblk
[root@rhel-8 ~]# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
xvda         202:0    0   10G  0 disk
├─xvda1      202:1    0    1M  0 part
└─xvda2      202:2    0   10G  0 part /
xvdb         202:16   0   10G  0 disk
xvdc         202:32   0   10G  0 disk
xvdd         202:48   0   10G  0 disk
xvde         202:64   0   10G  0 disk
xvdf         202:80   0   10G  0 disk
```

None of these block devices should have a partition table. You can confirm this using the command.

```
# blkid -p /dev/xvdb
[root@rhel-8 ~]#
[root@rhel-8 ~]# blkid -p /dev/xvdb
[root@rhel-8 ~]#
[root@rhel-8 ~]# blkid -p /dev/xvdc
[root@rhel-8 ~]#
[root@rhel-8 ~]# blkid -p /dev/xvdd
[root@rhel-8 ~]#
[root@rhel-8 ~]# blkid -p /dev/xvde
[root@rhel-8 ~]#
[root@rhel-8 ~]# blkid -p /dev/xvdf
[root@rhel-8 ~]#
```

If you get no output, then it means that your block devices do not have any partition table residing on them. However, in the event that a partition table exists, you can wipe it using the command:

```
# wipefs -a /<device-path>
```

Create a Stratis Pool from One Block Device

You can create a **Stratis** pool from a single block device using the syntax.

```
# stratis pool create <pool-name> <block-device>
```

For example to create a pool from `/dev/xvdb` run the command.

```
# stratis pool create my_pool_1 /dev/xvdb
```

To confirm the created pool run.

```
# stratis pool list
[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis pool create my_pool_1 /dev/xvdb
[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis pool list
Name                Total Physical Size  Total Physical Used
my_pool_1            10 GiB                52 MiB
[root@rhel-8 ~]#
```

Create a Stratis Pool from Multiple Block Devices

To create a pool from multiple devices, use the syntax below listing all the devices on one line.

```
# stratis pool create <pool_name> device-1 device-2 device-n
```

To create a pool from `/dev/xvdc`, `/dev/xvdd` and `/dev/xvde` run the command.

```
# stratis pool create my_pool_2 /dev/xvdc /dev/xvdd /dev/xvde
```

Once again, list the pools available using the command.

```
# stratis pool list
```

At this point, you should have 2 pools: **my_pool_1** and **my_pool_2**.

```
[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis pool create my_pool_2 /dev/xvdc /dev/xvdd /dev/xvde
[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis pool list
Name                Total Physical Size  Total Physical Used
my_pool_1            10 GiB                52 MiB
my_pool_2            30 GiB                60 MiB
[root@rhel-8 ~]#
```

As you can see above, the hard disk space occupied by pool **my_pool_2** is thrice that of the first pool which we created from only one block device with a memory of **10GB**.

Create a Filesystem from a Pool

Having created your filesystem, you can create a filesystem from one of the pools using the syntax.

```
# stratis fs create <poolname> <filesystemname>
```


For instance, to create **filesystem-1** and **filesystem-2** from **my_pool_1** and **my_pool_2** respectively run commands:

```
# stratis fs create my_pool_1 filesystem-1
# stratis fs create my_pool_2 filesystem-2

[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis fs create my_pool_1 filesystem-1
[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis fs create my_pool_2 filesystem-2
[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis fs create my_pool_2 filesystem-3
[root@rhel-8 ~]#
[root@rhel-8 ~]#
```

To view the newly created filesystems, run the command.

```
# stratis fs list

[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis fs list
Pool Name  Name          Used      Created          Device                                UUID
my_pool_1  filesystem-1  545 MiB   Oct 25 2019 11:03 /stratis/my_pool_1/filesystem-1      8a29e41073
my_pool_2  filesystem-2  545 MiB   Oct 25 2019 11:04 /stratis/my_pool_2/filesystem-2      014875aaa2
my_pool_2  filesystem-3  545 MiB   Oct 25 2019 11:04 /stratis/my_pool_2/filesystem-3      0002268461
[root@rhel-8 ~]#
```

To narrow down the results of a file system to one pool, run the command:

```
# stratis fs list <poolname>
```

For instance, to check the filesystem in **my_pool_2** run the command.

```
# stratis fs list my_pool_2

[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis fs list my_pool_2
Pool Name  Name          Used      Created          Device                                UUID
my_pool_2  filesystem-2  545 MiB   Oct 23 2019 23:59 /stratis/my_pool_2/filesystem-2      b485ce80be
[root@rhel-8 ~]#
```

Now, if you run the [lsblk command](#), the output should be somewhat similar to the sample output below.

```
# lsblk
```

```

[root@rhel-8 ~]#
[root@rhel-8 ~]# lsblk
NAME
xvda
├─xvda1
└─xvda2
xvdb
├─stratis-l-private-7be933b22d7d4bc4alcc4lccba80fbc-physical-originsub
│   └─stratis-l-private-7be933b22d7d4bc4alcc4lccba80fbc-flex-thinmeta
│       └─stratis-l-private-7be933b22d7d4bc4alcc4lccba80fbc-thinpool-pool
│           └─stratis-l-7be933b22d7d4bc4alcc4lccba80fbc-thin-fs-c632dcf53e2346c882b6b06a4cc9d6a7
├─stratis-l-private-7be933b22d7d4bc4alcc4lccba80fbc-flex-thindata
│   └─stratis-l-private-7be933b22d7d4bc4alcc4lccba80fbc-thinpool-pool
│       └─stratis-l-7be933b22d7d4bc4alcc4lccba80fbc-thin-fs-c632dcf53e2346c882b6b06a4cc9d6a7
└─stratis-l-private-7be933b22d7d4bc4alcc4lccba80fbc-flex-mdv
xvdc
├─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-physical-originsub
│   └─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-flex-thinmeta
│       └─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-thinpool-pool
│           └─stratis-l-7dlf77c9da6c46el93a39c97de9b246d-thin-fs-b485ce80bel84a068631925132bbfd78
├─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-flex-thindata
│   └─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-thinpool-pool
│       └─stratis-l-7dlf77c9da6c46el93a39c97de9b246d-thin-fs-b485ce80bel84a068631925132bbfd78
└─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-flex-mdv
xvdd
├─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-physical-originsub
│   └─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-flex-thinmeta
│       └─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-thinpool-pool
│           └─stratis-l-7dlf77c9da6c46el93a39c97de9b246d-thin-fs-b485ce80bel84a068631925132bbfd78
├─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-flex-thindata
│   └─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-thinpool-pool
│       └─stratis-l-7dlf77c9da6c46el93a39c97de9b246d-thin-fs-b485ce80bel84a068631925132bbfd78
└─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-flex-mdv
xvde
├─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-physical-originsub
│   └─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-flex-thinmeta
│       └─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-thinpool-pool
│           └─stratis-l-7dlf77c9da6c46el93a39c97de9b246d-thin-fs-b485ce80bel84a068631925132bbfd78
├─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-flex-thindata
│   └─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-thinpool-pool
│       └─stratis-l-7dlf77c9da6c46el93a39c97de9b246d-thin-fs-b485ce80bel84a068631925132bbfd78
└─stratis-l-private-7dlf77c9da6c46el93a39c97de9b246d-flex-mdv
[root@rhel-8 ~]# [root@rhel-8 ~]# lsblk

```

Mounting a Stratis Filesystem

We are now going to mount the existing filesystems in order to access them. First, create the mount points.

For the filesystem in the first pool, run the command:

```

# mkdir /data
# mount /stratis/my_pool_1/filesystem-1 /data

```

For the second filesystem in the second pool, run the command.

```

# mkdir /block
# mount /stratis/my_pool_2/filesystem-2 /block

```

```
[root@rhel-8 ~]#
[root@rhel-8 ~]# mkdir /data
[root@rhel-8 ~]#
[root@rhel-8 ~]# mount /stratis/my_pool_1/filesystem-1 /data
[root@rhel-8 ~]#
[root@rhel-8 ~]#
[root@rhel-8 ~]# mkdir /block
[root@rhel-8 ~]#
[root@rhel-8 ~]# mount /stratis/my_pool_2/filesystem-2 /block
[root@rhel-8 ~]#
```

To verify the existence of the current mount points run [df command](#):

```
# df -Th | grep stratis
[root@rhel-8 ~]#
[root@rhel-8 ~]# df -Th | grep stratis
/dev/mapper/stratis-l-7be933b22d7d4bc4alcc4lccbaf80fbc-thin-fs-c632dcf53e2346c882b6b06a4cc9d6a7
/dev/mapper/stratis-l-7dlf77c9da6c46el93a39c97de9b246d-thin-fs-b485ce80bel84a06863l925l32bbfd78
[root@rhel-8 ~]#
[root@rhel-8 ~]#
```

Perfect! We can clearly see that our mount points are present.

Persistently Mount Stratis Filesystems

The mount points that we have just created cannot survive a reboot. To make them persistent, first obtain the **UUID** of each of the filesystems:

```
# blkid -p /stratis/my_pool_1/filesystem-1
# blkid -p /stratis/my_pool_2/filesystem-2
[root@rhel-8 ~]#
[root@rhel-8 ~]# blkid -p /stratis/my_pool_1/filesystem-1
/stratis/my_pool_1/filesystem-1: UUID="c632dcf5-3e23-46c8-82b6-b06a4cc9d6a7" TYPE="xfs" USAGE="f
[root@rhel-8 ~]#
[root@rhel-8 ~]#
[root@rhel-8 ~]# blkid -p /stratis/my_pool_2/filesystem-2
/stratis/my_pool_2/filesystem-2: UUID="b485ce80-bel8-4a06-863l-925l32bbfd78" TYPE="xfs" USAGE="f
[root@rhel-8 ~]#
[root@rhel-8 ~]#
```

Now proceed and copy the UUID's and mount point options to **/etc/fstab** as shown.

```
# echo "UUID=c632dcf5-3e23-46c8-82b6-b06a4cc9d6a7 /data xfs defaults 0 0" | sudo tee -a
/etc/fstab
# echo "UUID=b485ce80-bel8-4a06-863l-925l32bbfd78 /block xfs defaults 0 0" | sudo tee -a
/etc/fstab
[root@rhel-8 ~]#
[root@rhel-8 ~]# echo "UUID=c632dcf5-3e23-46c8-82b6-b06a4cc9d6a7 /data xfs defaults 0 0" | sudo
UUID=c632dcf5-3e23-46c8-82b6-b06a4cc9d6a7 /data xfs defaults 0 0
[root@rhel-8 ~]#
[root@rhel-8 ~]# echo "UUID=b485ce80-bel8-4a06-863l-925l32bbfd78 /block xfs defaults 0 0" | sudo
UUID=b485ce80-bel8-4a06-863l-925l32bbfd78 /block xfs defaults 0 0
[root@rhel-8 ~]#
```

For the system to register the new configuration run the command:

```
# systemctl daemon-reload
```


To verify the configuration works as expected, mount the filesystems.

```
# mount /data
# mount /block
```

Removing a Stratis Filesystem

To remove a filesystem, you need to, first of all, unmount the file system as shown.

```
# umount /mount-point
```

In this case, we shall have.

```
# umount /data
```

To destroy the filesystem, use the syntax:

```
# stratis filesystem destroy <poolname> <filesystem-name>
```

So, we shall have:

```
# stratis filesystem destroy my_pool_1 filesystem-1
[root@rhel-8 ~]# umount /data
[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis filesystem destroy my_pool_1 filesystem-1
[root@rhel-8 ~]#
```

To confirm the removal of the filesystem, issue the command.

```
# stratis filesystem list my_pool_1
[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis filesystem list my_pool_1
Pool Name  Name  Used  Created  Device  UUID
[root@rhel-8 ~]#
```

From the output, we can clearly see that the filesystem associated with **my_pool_1** has been deleted.

Adding a Disk to an Existing Stratis Pool

You can add a disk to an existing pool using the command:

```
# stratis pool add-data <poolname> /<devicepath>
```

For example, to add an additional disk `/dev/xvdf`, to **my_pool_1**, run the command:

```
# stratis pool add-data my_pool_1 /dev/xvdf
[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis pool add-data my_pool_1 /dev/xvdf
[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis pool list
Name                Total Physical Size  Total Physical Used
my_pool_1            20 GiB                56 MiB
my_pool_2            30 GiB                605 MiB
[root@rhel-8 ~]#
```

Notice that the size of **my_pool_1** has double in size after adding the extra volume.

Create Stratis Snapshots

A snapshot is a thinly provisioned read and writes a copy of a filesystem at a given point in time.

To create a snapshot, run the command:

```
# stratis fs snapshot <poolname> <fsname> <snapshotname>
```

In this case, the command will be:

```
# stratis fs snapshot my_pool_2 filesystem-2 mysnapshot
```

You can append the data attribute `-${date +%Y-%m-%d}` to the snapshot add the date tag as shown in the screenshot below.

To verify the creation of the snapshot, run the command:

```
# stratis filesystem list <poolname>
```

In this case, the command will be:

```
# stratis filesystem list my_pool_2
```

```
[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis fs snapshot my_pool_2 filesystem-2 mysnapshot-$(date +%Y-%m-%d)
[root@rhel-8 ~]#
[root@rhel-8 ~]#
[root@rhel-8 ~]# stratis filesystem list my_pool_2
Pool Name  Name                Used      Created          Device
  UUID
my_pool_2  mysnapshot-2019-10-24 545 MiB   Oct 24 2019 20:46 /stratis/my_pool_2/mysnapshot-2019
-24 075544c5fc3349988e566e0cc279d92d
my_pool_2  filesystem-2         545 MiB   Oct 23 2019 23:59 /stratis/my_pool_2/filesystem-2
b485ce80bel84a068631925132bbfd78
[root@rhel-8 ~]#
```

Reverting to a Previous Snapshot Using a Filesystem

To revert a Stratis filesystem to a previously created snapshot, first, unmount and destroy the original filesystem.

```
# umount /stratis/<poolname>/filesystem
```

In our scenario, this will be.

```
# umount /stratis/my_pool_2/filesystem-2
```

Then create a copy of the snapshot using the original file system:

```
# stratis filesystem snapshot <poolname> filesystem-snapshot filesystem
```

The command will be:

```
# stratis filesystem snapshot my_pool_2 mysnapshot-2019-10-24 block
```

Finally, mount the snapshot.

```
# mount /stratis/my-pool/my-fs mount-point
```

Removing a Stratis Snapshot

To remove the snapshot, first, unmount the snapshot.

```
# unmount /stratis/my_pool_2/mysnapshot-2019-10-24
```

```
[root@rhel-8 ~]#  
[root@rhel-8 ~]# umount /stratis/my_pool_2/mysnapshot-2019-10-24  
[root@rhel-8 ~]#
```

Next, proceed and destroy the snapshot:

```
# stratis filesystem destroy my_pool_2 mysnapshot-2019-10-24
```

```
[root@rhel-8 ~]#  
[root@rhel-8 ~]# stratis filesystem destroy my_pool_2 mysnapshot-2019-10-24  
[root@rhel-8 ~]#
```

Removing a Stratis Pool

To remove a Stratis pool, follow the simple steps below.

1. List the filesystems that exist in the pool.

```
# stratis filesystem list <poolname>
```

2. Next, unmount all the filesystems in the pool.

```
# umount /stratis//filesystem-1  
# umount /stratis//filesystem-2  
# umount /stratis//filesystem-3
```

3. Destroy the filesystems.

```
# stratis filesystem destroy <poolname> fs-1 fs-2
```

4. And then, get rid of the pool.

```
# stratis pool destroy poolname
```

In this case, the syntax will be.

```
# stratis pool destroy my_pool_2
```

```
[root@rhel-8 ~]#  
[root@rhel-8 ~]# stratis pool destroy my_pool_2  
[root@rhel-8 ~]#  
[root@rhel-8 ~]# stratis pool list  
Name                Total Physical Size  Total Physical Used  
my_pool_1            20 GiB                56 MiB  
[root@rhel-8 ~]#
```

You can verify the pool list again.

```
# stratis pool list
```

Finally, remove the entries in the **/etc/fstab** for the filesystems.

VDO

VDO short for **Virtual Data Optimizer** is a block virtualization technology that provides inline deduplication and compression of data at a block device level.

The idea of deduplication is quite simple: to remove copies of duplicate data and only remain with one copy. When an identical file is added on a block device, it is marked as a duplicate and the original file is referenced instead. In so doing, **VDO** helps in saving the block volume's space.

In this tutorial, you will learn how to create a **VDO** volume on a storage device on the **RHEL 8** system.

Step 1: Install the VDO in RHEL 8

To get started, log in to your server and update your **RHEL** using the [dnf command](#).

```
$ sudo yum update -y
```

```
[tecmint@rhel-8 ~]$  
[tecmint@rhel-8 ~]$ sudo dnf update -y  
Last metadata expiration check: 0:06:59 ago on Fri 24 Jan 2020 12:31:23 AM UTC.  
Dependencies resolved.
```

Package	Arch	Version	Repository	Size
Installing:				
yum-utils	noarch	4.0.8-3.el8	rhui-rhel-8-baseos-rhui-rpms	64 k
replacing dnf-utils.noarch	noarch	4.0.2.2-3.el8		
kernel	x86_64	4.18.0-147.3.1.el8_1	rhui-rhel-8-baseos-rhui-rpms	1.5 M
kernel-modules	x86_64	4.18.0-147.3.1.el8_1	rhui-rhel-8-baseos-rhui-rpms	22 M
kernel-core	x86_64	4.18.0-147.3.1.el8_1	rhui-rhel-8-baseos-rhui-rpms	25 M
Upgrading:				
rh-amazon-rhui-client	noarch	3.0.23-1.el8	rhui-client-config-server-8	35 k
python3-rhn-check	x86_64	2.8.16-13.module+el8.1.0+3455+3ddf2832	rhui-rhel-8-appstream-rhui-rpms	38 k
cloud-init	noarch	18.5-7.el8	rhui-rhel-8-appstream-rhui-rpms	856 k
python3-rhnlib	noarch	2.8.6-8.module+el8.1.0+3455+3ddf2832	rhui-rhel-8-appstream-rhui-rpms	77 k

After the update of the packages & kernel is complete, proceed and install the **VDO** kernel modules and dependencies using the command.

```
$ sudo dnf install kmod-kvdo vdo
```


- **vdo** – This is a set of Management tools for Virtual Data Optimizer.
- **kmod-kvdo** – This is a group of Kernel Modules for Virtual Data Optimizer.

```
[tecmint@rhel-8 ~]$
[tecmint@rhel-8 ~]$ sudo dnf install kmod-kvdo vdo
[sudo] password for tecmint:
Red Hat Update Infrastructure 3 Client Configuration Server 8 14 kB/s | 2.1 kB 00:00
Red Hat Enterprise Linux 8 for x86_64 - AppStream from RHUI ( 27 MB/s | 14 MB 00:00
Red Hat Enterprise Linux 8 for x86_64 - BaseOS from RHUI (RPM 28 MB/s | 13 MB 00:00
Dependencies resolved.
```

Package	Arch	Version	Repository	Size
Installing:				
vdo	x86_64	6.2.1.134-11.el8	rhel-8-baseos-rhui-rpms	689 k
kmod-kvdo	x86_64	6.2.1.138-57.el8	rhel-8-baseos-rhui-rpms	355 k
Installing dependencies:				
device-mapper-event-libs	x86_64	8:1.02.163-5.el8	rhel-8-baseos-rhui-rpms	265 k
libaio	x86_64	0.3.112-1.el8	rhel-8-baseos-rhui-rpms	33 k
device-mapper-event	x86_64	8:1.02.163-5.el8	rhel-8-baseos-rhui-rpms	266 k
lvm2	x86_64	8:2.03.05-5.el8	rhel-8-baseos-rhui-rpms	1.5 M
device-mapper-persistent-data	x86_64	0.8.5-2.el8	rhel-8-baseos-rhui-rpms	473 k
lvm2-libs	x86_64	8:2.03.05-5.el8	rhel-8-baseos-rhui-rpms	1.1 M

```
Transaction Summary
=====
Install 8 Packages

Total download size: 4.6 M
Installed size: 16 M
Is this ok [y/N]: y
```

After successful installation, start, enable and verify the **vdo** daemon.

```
$ sudo systemctl start vdo
$ sudo systemctl enable vdo
$ sudo systemctl status vdo

[tecmint@rhel-8 ~]$
[tecmint@rhel-8 ~]$ sudo systemctl status vdo.service
● vdo.service - VDO volume services
   Loaded: loaded (/usr/lib/systemd/system/vdo.service; enabled; vendor preset: enabled)
   Active: active (exited) since Fri 2020-01-24 00:51:18 UTC; 1min 10s ago
 Main PID: 24630 (code=exited, status=0/SUCCESS)
    Tasks: 0 (limit: 11449)
   Memory: 0B
    CGroup: /system.slice/vdo.service

Jan 24 00:51:17 rhel-8 systemd[1]: Starting VDO volume services...
Jan 24 00:51:18 rhel-8 systemd[1]: Started VDO volume services.
[tecmint@rhel-8 ~]$
```

Step 2: Create a VDO Volume in RHEL 8

Before creating a **vdo** volume, ensure that you have an additional hard drive on your system. In this tutorial, we have attached an additional volume `xvdb`. This can be demonstrated by running the [lsblk command](#) below.

```
$ lsblk
```

```
[tecmint@rhel-8 ~]$
[tecmint@rhel-8 ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
xvda         202:0    0   20G  0 disk
├─xvda1      202:1    0    1M  0 part
├─xvda2      202:2    0   20G  0 part /
xvdb         202:16   0  100G  0 disk
```

From the output, you can clearly see that the second disk has a capacity of **100GB**.

Now, we shall create an empty **VDO** volume on `/dev/xvdb` disk.

```
$ sudo vdo create --name=vdol --device=/dev/xvdb --vdoLogicalSize=300G
```

You will encounter the error shown.

```
[tecmint@rhel-8 ~]$
[tecmint@rhel-8 ~]$ sudo vdo create --name=vdol --device=/dev/xvdb --vdoLogicalSize=300G
Creating VDO vdol
vdo: ERROR - Kernel module kvdo not installed
vdo: ERROR - modprobe: FATAL: Module kvdo not found in directory /lib/modules/4.18.0-80.4.2.el
8_0.x86_64
[tecmint@rhel-8 ~]$
```

This is a common bug and the workaround is to simply reboot your server.

```
$ sudo reboot
```

On the second trial, the command will be executed, creating an empty **VDO** volume on the device `/dev/xvdb`.

```
$ sudo vdo create --name=vdol --device=/dev/xvdb --vdoLogicalSize=300G
[tecmint@rhel-8 ~]$
[tecmint@rhel-8 ~]$ sudo vdo create --name=vdol --device=/dev/xvdb --vdoLogicalSize=300G
Creating VDO vdol
Starting VDO vdol
Starting compression on VDO vdol
VDO instance 0 volume is ready at /dev/mapper/vdol
[tecmint@rhel-8 ~]$
[tecmint@rhel-8 ~]$
```

Let's break down the command and have a look at the option used:

- **create** – This initiates the creation of the VDO volume.
- **--name=vdol** – This gives the volume a label known as **vdol**. Feel free to assign any name of your choice.
- **--device=/dev/xvdb** – The device option specifies the disk on which the volume will be created.
- **--vdoLogicalSize=300G** – This indicates the effective volume capacity to be used by the operating system, in this case, **300G**.

Step 3: Probing the New VDO Volume

The new **VDO** volume has been created at `/dev/mapper/vdol` going by the output we saw in the previous step. We can use the [ls command](#) as shown to investigate file permissions & ownership.

```
$ ls -l /dev/mapper/vdol
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ ls -l /dev/mapper/vdol
lrwxrwxrwx. 1 root root 7 Jan 24 19:59 /dev/mapper/vdol -> ../dm-0
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$
```

To obtain more insightful information use the **vdostats** command to retrieve statistics on the size and the usage of the volume.

```
$ vdostats --hu
```

The `--hu` flag displays the information in a human-readable format, i.e. a format that is much easier to read and decipher with ease. We can view attributes such as the Device name, the size on the additional disk, used and available space as the % usage.

```
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo vdostats --hu
Device          Size      Used Available Use% Space saving%
/dev/mapper/vdol 100.0G    4.1G    95.9G     4%      N/A
[tecmint@rhel8 ~]$
```

Carefully observe that the % Saving is indicated as **Not Applicable (N/A)**.

Also, note that already we have some volume usage **4.1G** which translates to **4%** yet we haven't written anything on the volume. Why is that? This is because the universal Deduplication has already been written onto the disk and it is what makes deduplication possible.

The **vdostats** command can be used with the `--verbose` flag to retrieve more detailed information as shown:

```
$ sudo vdostats --verbose /dev/mapper/vdol | grep -B6 'saving percent'
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo vdostats --verbose /dev/mapper/vdol | grep -B6 'saving percent'
physical blocks      : 26214400
logical blocks       : 78643200
1K-blocks            : 104857600
1K-blocks used       : 4262272
1K-blocks available  : 100595328
used percent         : 4
saving percent       : N/A
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$
```

What you can see is basically the same data as the previous example but in a different format.

Step 4: Partitioning the VDO Volume

After obtaining enough insights from the volume, we need to partition it and later create a filesystem so that it can be used as a normal disk.

Let's create a physical volume and volume group as shown, run the following commands.

```
$ sudo pvcreate /dev/mapper/vd01
$ sudo vgcreate vdolvg /dev/mapper/vd01
```

```
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo pvcreate /dev/mapper/vd01
Physical volume "/dev/mapper/vd01" successfully created.
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo vgcreate vdolvg /dev/mapper/vd01
Volume group "vdolvg" successfully created
[tecmint@rhel8 ~]$
```

To display the statistics of the volume group run:

```
$ sudo vgdisplay vdolvg
```

```
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo vgdisplay vdolvg
--- Volume group ---
VG Name                vdolvg
System ID
Format                 lvm2
Metadata Areas         1
Metadata Sequence No   1
VG Access               read/write
VG Status               resizable
MAX LV                 0
Cur LV                 0
Open LV                 0
Max PV                  0
Cur PV                 1
Act PV                  1
VG Size                 <300.00 GiB
PE Size                 4.00 MiB
Total PE                76799
Alloc PE / Size         0 / 0
Free PE / Size          76799 / <300.00 GiB
VG UUID                 It2xhe-OdKt-J2TS-PcVi-lpul-vKf8-mKsTlO
```

Now, we will create 2 equally sized logical volumes each with a capacity of **50G**.

```
$ sudo lvcreate -n vdolv01 -L 50G vdolvg
$ sudo lvcreate -n vdolv02 -L 50G vdolvg
```

```
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo lvcreate -n vdolv01 -L 50G vdolvg
Logical volume "vdolv01" created.
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo lvcreate -n vdolv02 -L 50G vdolvg
Logical volume "vdolv02" created.
[tecmint@rhel8 ~]$
```

You can later view the stats of the newly created volumes by running the command.

```
$ sudo lvs
```



```
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo lvs
  LV      VG      Attr      LSize   Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  vdolv01 vdolvg -wi-a----- 50.00g
  vdolv02 vdolvg -wi-a----- 50.00g
[tecmint@rhel8 ~]$
```

Step 4: Formatting and Mounting File Systems

Usually, when a filesystem is created, a trim operation is carried out on the device. This is undesirable in the case of the **VDO**. When formatting using the **mkfs** command, use the **-K** option to instruct the command not to discard blocks during the creation of the filesystem.

```
$ sudo mkfs.xfs -K /dev/vdolvg/vdolv01
$ sudo mkfs.xfs -K /dev/vdolvg/vdolv02
```

If you are using **EXT4** filesystem, use the “**-E nodiscard**” Option.

Create the mount points for mounting the volumes:

```
$ sudo mkdir /data/v01
$ sudo mkdir /data/v02
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo mkdir /data/v01
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo mkdir /data/v02
[tecmint@rhel8 ~]$
```

Now mount the filesystems onto their mount points as shown.

```
$ sudo mount -o discard /dev/vdolvg/vdolv01 /data/v01
$ sudo mount -o discard /dev/vdolvg/vdolv02 /data/v02
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo mount -o discard /dev/vdolvg/vdolv01 /data/v01
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo mount -o discard /dev/vdolvg/vdolv02 /data/v02
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$
```

Now when you inspect the **VDO** volume you’ll notice that the % saving has changed to **99%** which is quite impressive. This implies that deduplication is working as expected.

```
$ sudo vdostats --hu
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$ sudo vdostats --hu
Device          Size      Used Available Use% Space saving%
/dev/mapper/vdol 100.0G    4.1G      95.9G      4%      99%
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$
[tecmint@rhel8 ~]$
```

You can further investigate using the [df -Th command](#). At the bottom section, you’ll see the filesystems mounted on /data/v01 and /data/v02 respectively.

```
$ df -hT
```

```
[tecmint@rhel8 ~]$  
[tecmint@rhel8 ~]$ df -Th  
Filesystem                Type      Size  Used Avail Use% Mounted on  
devtmpfs                  devtmpfs  1.9G   0    1.9G   0% /dev  
tmpfs                     tmpfs     1.9G   0    1.9G   0% /dev/shm  
tmpfs                     tmpfs     1.9G  17M   1.9G   1% /run  
tmpfs                     tmpfs     1.9G   0    1.9G   0% /sys/fs/cgroup  
/dev/xvda2                 xfs       20G  1.9G   19G  10% /  
tmpfs                     tmpfs     378M   0    378M   0% /run/user/1000  
/dev/mapper/vdolv-g-vdolv01 xfs       50G  390M   50G   1% /data/v01  
/dev/mapper/vdolv-g-vdolv02 xfs       50G  390M   50G   1% /data/v02  
[tecmint@rhel8 ~]$
```

Eng. Muhammad Adel